Lab Assignments
Computer Graphics DT519A,
Örebro University, autumn 2024

November 6, 2024

2 Materials, reflectance and light

In this lab you will implement and experiment with different material models.

You will set up a framework for the appearance of materials in your fragment shader and implement some physically-based material models, and experiment with how to use them to render different kinds of materials, as well as the limits and virtues of the different models.

Learning goals

- Understanding the principles of material reflectance models.
- Understanding the effects of the parameters of common models.
- Managing normal-vector vertex attributes in OpenGL.

Setup

Start by downloading the code skeletons in `Lab2.zip` and extract the contents in your existing folder structure from the previous lab.

In this lab, you will mainly work with the fragment shader. You can start by copying your final vertex shader from Lab 1.

Mandatory 2.1 Points with normals

To begin with, you will define the surface normal direction of each side of your cube from Lab 1. In addition to the *position* vertex attribute that you used in Lab 1, you will now also need to specify the *normal vector* as an attribute for each vertex.

4D vs 3D From this point on, we will explicitly send 4D vectors to the vertex shader. Note in the skeleton code that the array with vertex positions now has 4 components per vertex: (x, y, z, w), with $w = 1$. Recall that $w = 1$ for points (i e, positions) and $w = 0$ for directions (i e, normals).

The vertex shader handles transformations with 4x4 matrix-vector multiplications and therefore needs to use 4D vectors. However, when passing `out` variables to the fragment shader, I recommend that you stick to `vec3`, so that all vectors are 3D in the fragment shader.[1]

[1] While it may be more consistent to use only 4D vectors throughout, and more memory-efficient to pass 3D vectors to the VS and handle the extra 4th coordinate internally there, separating 4D vs 3D with this convention is likely to give you the least headaches. This way, it is clear in your C++ code that you separate directions from points with the w coordinate. Since the fragment shader does not need to do any transformations, the 4th coordinate is not needed there, so simply omitting it makes sure that an erroneous w-coordinate does not cause trouble. (Otherwise, a common mistake is to try to normalise a vector with $w = 1$ which will give you the wrong vector length, for example.) On the other hand, working with 3D vectors in the vertex shader would lead to other,

Duplicate vertices Before, you were using only 8 vertices, together with an index array to generate all the 12 triangular faces of the cube object. But now you will have to use $3 \times 8 = 24$ vertices, because the normal at a vertex depends on which face it belongs to, and a vertex cannot have more than one normal. (See also Figure 1.) Concretely, you will need to set up *three* vertices for each corner of the cube, each with the same position but with different normals, since the corner is shared between three faces with one normal each. In other words, the array of vertex positions now needs 3×8 vertices (with 3 copies of each of your vertices from before). The number of faces are still the same (12), but because we now have more vertices, the indices in the EBO need to change. For convenience, this geometry is defined in the skeleton code.

Note that this duplication of vertices is needed only for objects with sharp corners, such as this cube. For smoother objects (like the Stanford bunny in Figure 7), with no discontinuities in the normal along the surface, this is not needed.

Tasks

Computing normals Once the geometry is set up, loop over all faces and compute the surface normal for each. Each triangle has one normal vector, and this normal vector should then be assigned to all three vertices making up the face. If you have your triangles defined such that the vertices $\vec{p}_1, \vec{p}_2, \vec{p}_3$ are ordered *counter-clockwise*, then the normal vector (pointing outwards) is given by $\vec{p}_2 - \vec{p}_1 \times \vec{p}_3 - \vec{p}_1$.[2] (The geometry in the skeleton code has the vertices ordered in this way.) Hint: it's convenient to use the `glm::cross` function for the cross product.

Having the normals pointing outwards is the only way of specifying what is the "inside" and "outside" of a triangle, and OpenGL may skip rendering triangles where only the back side is visible, for efficiency reasons. You can store the normals on the CPU side in a `std::vector` or plain array of `glm::vec4`s. Ensure that the w coordinate of the normals you have computed is 0 (because it is a direction and not a point). The normal vector computed for each face must then be stored for each of its three connected vertices.

At this point, it makes sense to print out for each vertex its position attribute and normal attribute. Since this is an axis-aligned cube, all normals should have one component exactly $+1$ or -1, and the other two exactly 0. If you get different results, you have done something wrong.

Buffer objects for more than one attribute When setting up your VBOs, you can either have one buffer object that stores both vertex positions and normals, or you can use a separate buffer object for each vertex attribute. If you use a single VBO, you can choose between interleaving the vertex attributes, or having all the positions first followed by all the normals, for example. This is potentially tricky. Look closely at the documentation for `glVertexArrayAttribFormat` and `glVertexArrayVertexBuffer` to see how to specify where and in which order the attributes are stored.

I recommend using separate VBOs for the position and normal attributes. Then, setting up the geometry means to (1) create the VAO, (2) create a VBO for

unfixable, troubles – like changing normal directions when an object is translated. Why did it ever work in Lab 1 then, when we passed attributes with 3 components each into a `vec4` on the vertex shader? It is because when GLSL constructs a `vec4` from less than 4 values, the missing values are filled in with $(0, 0, 0, 1)$.

[2] See also https://www.opengl.org/wiki/Calculating_a_Surface_Normal

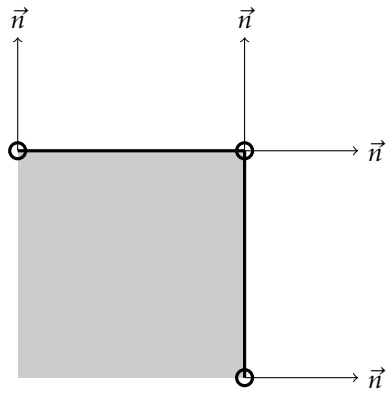


Figure 1: Viewing the cube object from the side. For the normals to be consistent over a flat surface, the vertices defining it must have the same normal. In this example, the corner vertex would need to have two normals, which is not possible. Instead, we'll need to duplicate the vertex and place two vertices at the same position, each with a different normal.

the position attribute, (3) buffer the data, (4) enable the attribute (i e, number 0), set the binding point to 0, (5) set up the vertex attrib format for attribute 0, (6) create a VBO for the normals, and buffer the data, (7) enable this attribute (number 1), set the binding point to 1, (8) set up its vertex attrib format, (9) set up the EBO as before.

Transformations to rasteriser vs fragment shader In the vertex shader, you should multiply the position that gets stored in `gl_Position` with the modelViewProjection matrix as before (for rasterisation). However, you will also need to pass on a version of the position and normal attributes for each vertex *without* the projection matrix, for computing the correct colour in the fragment shader. Since the projection matrix re-scales the z coordinate, you will otherwise have the wrong depth coordinates when computing the light angles etc in the fragment shader. In other words, the rasteriser works in normalised device coordinates (clip space), but the light computations in the fragment shader must happen in the view space (or world space)!

Colour from normal In your fragment shader, set the colour according to the normal-vector attribute. This makes it easier to debug that you have the right values assigned to each vertex. Since some normals will be pointing in negative directions, and negative values for RGB will produce a black colour, a sensible mapping from XYZ to RGB is `frag_colour.r = (normal.x+1)*0.5`

The result should look as in Figure 2, and the colours should change when the cube is rotated, since the normal vectors also rotate when the surfaces rotate. Surfaces that face to the right should be more red, and those that point up should be more green.

Mandatory

2.2 Your first BRDF

Now you will add a basic material model to your object, and to model how light is reflected off its surfaces. More precisely, step one will be to implement a Lambertian BRDF, which should provide the effect of "flat"/matte paint.

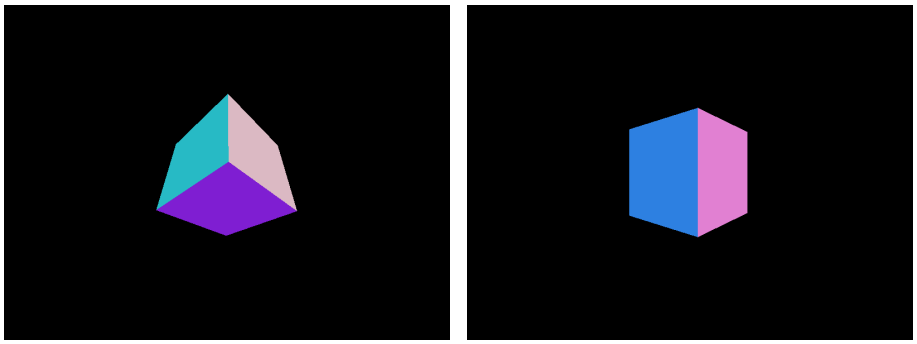


Figure 2: Output from Lab 2.1 – a cube with flat faces. Note that each face of the cube has one solid colour (no gradient), and that the colour changes with the orientation of the surface.

A central part of the general rendering equation is the following equation that describes how much light is reflected off a surface in a specific direction as a function of the incoming light and the surface normal (see also Lecture 3).

$$L_{\text{out}}(\vec{\omega}_{\text{o}}) = \underbrace{f_{\text{r}}(\vec{\omega}_{\text{i}}, \vec{\omega}_{\text{o}})}_{\text{BRDF}} \cdot \overbrace{L_{\text{in}}(\vec{\omega}_{\text{i}})}^{\text{incoming irradiance}} \cdot \underbrace{(\vec{\omega}_{\text{i}} \cdot \vec{n})}_{\text{incident angle}} \tag{1}$$

In the above equation, $\vec{\omega}_{\text{i}}$ and $\vec{\omega}_{\text{o}}$ are the directions from a surface point to the light source and the camera, respectively; $\vec{n}$ is the surface normal at the point; and f_{r} is the BRDF at the point.

Please note that the last dot in Equation (1) is the dot product of two vectors, while the other two are ordinary scalar multiplication. The `*` operator for vectors in GLSL does element-wise multiplication, which is something very different. You will need to use the `dot` function for dot products in GLSL.

We will approximate the integral of the rendering equation with summing up the output of one such equation per light source.

Tasks

A Lambert-shaded cube First, you will need to instantiate a point light source (or several, if you want). Each light source should have a position vector and a colour vector. For now, let's say that a bright white light has colour $[1, 1, 1]$ without a unit. Pass them to the shader program with `uniform`s (i e, not vertex attributes, since the light source data is the constant for all vertices). That should be all on the CPU side of the program.

Please note that the provided skeleton fragment shader `lab2-2_fs.glsl` defines three `uniform` input variables in order to handle multiple light sources: `light_count` and two arrays of `vec3`s for the positions and colours of the light sources. Recall that to set an integer `uniform` variable, you should call `glUniform1i` on the CPU side. To set a `vec3` array, you should use `glUniform3fv`. To simplify things, it is also perfectly fine to set `light_count` to 1, and have a single `vec3` for its position and colour!

For each light source (inside the foor loop), you will need to multiply the light that comes from the light source (the position and colour/intensity of the light should be provided with uniforms) with the BRDF at this fragment's position (the position gets passed on from the vertex shader) and the cosine

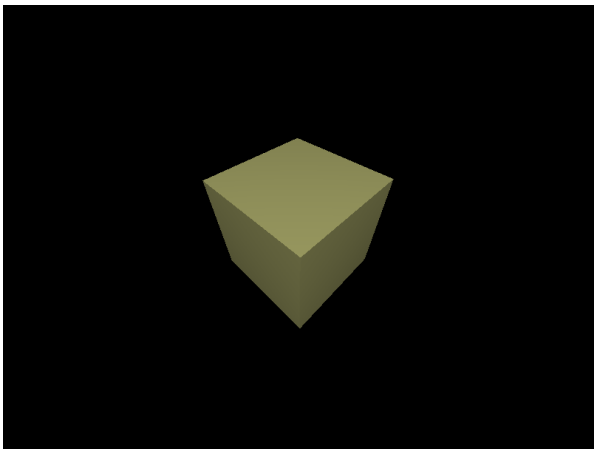


Figure 3: Output from task 2.2 – a cube with Lambertian shading. In this case, there is a light source above the object, at position (0,5,2). The yellow colouring is optional!

between the normal vector and the unit vector pointing to the light (which you compute here in the fragment shader). The only output direction $\vec{\omega}_o$ of interest is *from* the surface point handled by this invocation of the fragment shader, *towards* the camera.

Make sure that all your vectors are normalised! This can be done by the built-in GLSL function `normalize`. Also make sure that you only consider light from angles within 90° of the normal (that is, not light from behind the surface). In other words, check that the dot products are not negative. Otherwise you may not get the expected result.

The fragment shader in the template code includes an empty BRDF evaluation function that takes three arguments and returns a colour. A simple Lambertian BRDF doesn't really need the arguments. However, all of the BRDF functions to be implemented later in this lab use the same interface.

See Chapter 9.3 in RTR4 for more details on BRDFs in general, including what a Lambertian BRDF looks like.

See Figure 3 for example output. Please note that the albedo ρ of your Lambertian BRDF does not need to be the same for all colour components (wavelengths). You can also have a 3D vector $\vec{\rho}$ with different values for R, G, and B, to get a non-white colour (albedo) as in this example.

Question 1 What value does your BRDF return for the red/green/blue channel? Why?

If you are being physically correct, it may be that your object looks quite dark. In that case, a simple fix is to increase the brightness of the light. (You can multiply it by 3.)

Plotting the BRDF Copy your BRDF function to a CPU-side program that includes a reimplementation of your GLSL `lambertian_brdf` function. E g, make a small C++ program that uses GLM. For this task, the BRDF function should return a single scalar, and not a colour vector. (I e, we're plotting it for a single colour component.) So the function head should look like this, if you use C++:

```
double lambertian_brdf( glm::vec3 in_direction,
                        glm::vec3 out_direction,
                        glm::vec3 normal )
```

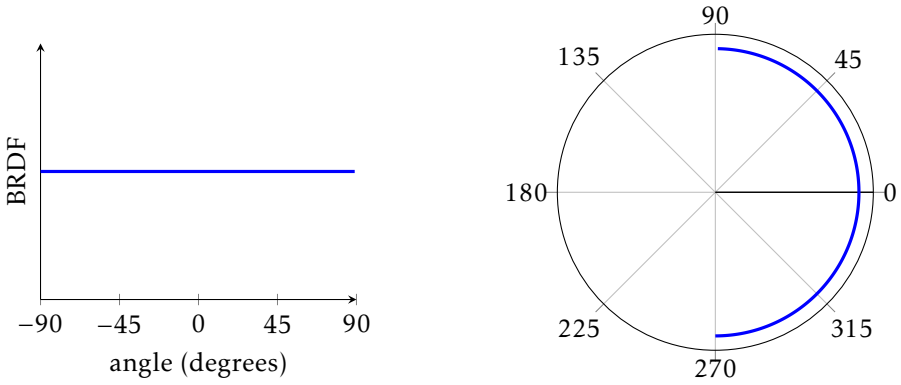


Figure 4: Plotting a Lambertian BRDF. *Left:* Cartesian coordinates. *Right:* polar coordinates.

Using your favourite plotting program, plot a 1D slice of your `lambertian_brdf` as follows. Set `normal` and `out_direction` to $(0, 0, 1)$, and make a loop to vary `in_direction` from $(1, 0, 0) = -90°$ to $(-1, 0, 0) = +90°$. Your output should look like Figure 4 (either the left or the right version).

Question 2 Include this plot in your report. Check that the plotted value is consistent with your answer to Question 1.

Mandatory

2.3 Blinn–Phong

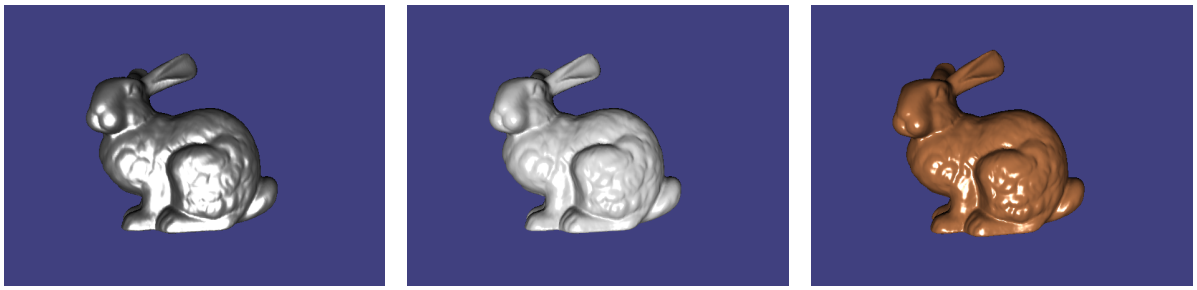
Loading a more detailed object It might be difficult to judge the difference in shading and highlights with an object with perfectly flat sides, like the cube that you have used so far. Update your code to load a more detailed 3D model from a file. To do this, use `tiny_obj_loader.h` that is provided with the skeleton code. In the lab directory (under common/data) there is also a model of the Stanford bunny.[3] Included is also a version of the Utah teapot.[4] You can use these, or some other model in Wavefront's `.obj` file format.

Note that you might have to scale up the bunny (try `glm::scale` with a factor 10), or move your camera closer, to see it better. For the teapot, you will have to scale it down quite a lot. Try to apply a `glm::scale` with a factor 0.05 to your model matrix. Note that if your model matrix includes scaling, the normals will *also* be scaled in the vertex shader, so you have to re-normalise them! Or, use a separate transformation matrix for the positions and the normals.

The bunny object is defined such that the origin is slightly below the bottom of the object (in the y direction). Therefore it may be convenient to define your model matrix such that you first transform the object with $(0, -1, 0)$.

Finally, there are some caveats to be aware of when buffering the data that is read by `tiny_obj_loader`, compared with your "manual" float array from before. In `glNamedBufferData` you have to say how many bytes to copy, but `sizeof(v)` with `v` being a `std::vector<float>` will not give you that size. Rather do something like `v.size()*sizeof(float)`. And you cannot get a pointer to with `&shapes[0].mesh.positions`, but `&shapes[0].mesh.positions[0]` will work. The same thing goes for all buffer objects: both VBOs and EBOs. Also note that `tiny_obj_loader` only reads three values for position and normal, not four. So you need to adjust your `glVertexArrayAttribFormat` accordingly.

[3] From the Stanford 3D Scanning Repository. See also a history of the bunny.
[4] Here's a bit of history about the teapot: https://en.wikipedia.org/wiki/Utah_teapot.



(a) $k_L = 0.5, k_g = 0.5, s = 10, \vec{\rho_L} = [1,1,1]$

(b) $k_L = 0.9, k_g = 0.1, s = 100, \vec{\rho_L} = [1,1,1]$

(c) $k_L = 0.9, k_g = 0.1, s = 100, \vec{\rho_L} = [0.8, 0.5, 0.3]$

Figure 5: Stanford bunnies with Blinn–Phong shading.

Implementing a Blinn–Phong BRDF One common BRDF model for glossy materials is the Blinn–Phong model, which is an adaptation of the classic Phong model that was at one time the only shading model that was available (in the fixed shading pipeline). The RTR book only explicitly lists the so-called normal-distribution function of Blinn–Phong (9.40), as part of a general microfacet model, but here it will be more convenient to write out Blinn–Phong as a complete BRDF:

$$f_r = k_L\left(\frac{\rho_L}{\pi}\right) + k_g\left(\rho_g \frac{8+s}{8\pi}(\vec{n}\cdot\vec{h})^s\right) \tag{2}$$

This equation has three parameters: k_L the amount of Lambertian scattering, k_g the amount of glossy scattering, and s the smoothness of the surface. Setting $k_g = 0$ will render a completely Lambertian surface. We should always have $k_L + k_g \leq 1$. Setting s to a small value (e.g., 1) will cause broad highlights, and large values (e.g., 1000) will produce sharp pointy highlights.

In addition, it has two colour components that control the hue of the surface. The ρ_L parameter is the same *albedo* as for a Lambertian function. In the equation above it is written as a scalar, but in your GLSL code it will rather be a vector with possibly different values for the r/g/b components. The ρ_g parameter controls the hue of the highlights. If it is "white" the highlights will take on the colour of the light source, which is what happens for non-metallic surfaces. If $\rho_g = \rho_L$ the highlights will have the same hue as the rest of the surface, which will make it appear more metallic.

Your task is to replace the Lambertian BRDF from the previous task with a Blinn–Phong BRDF and render your object with some different parameters. In other words, write a function

```
vec3 blinn_phong_brdf( vec3 in_direction,
                       vec3 out_direction,
                       vec3 normal )
```

in the fragment shader, and call that instead of `lambertian_brdf` from where you compute the reflectance equation (keeping the factors for incoming light and cosine the same).

See Figure 5 for example output. Note that in Figures 5 and 7, I have scaled up the light source by π to make things less dark.

Question 3 **Plotting the BRDF** Similar as in the previous task, plot a 1D version of your Blinn–Phong BRDF too, and include it in the report. It should look as in Figure 6 if you use the same parameters.

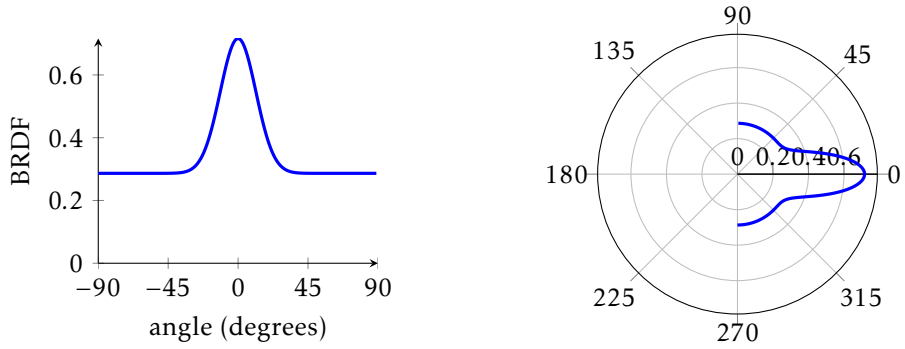
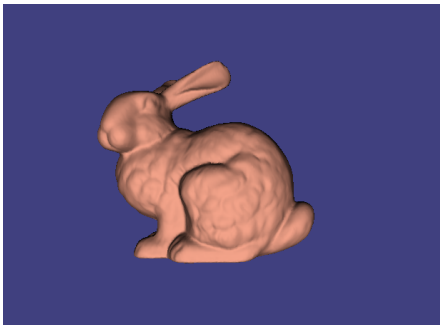


Figure 6: Plotting a Blinn–Phong BRDF. *Left:* Cartesian coordinates. *Right:* polar coordinates. $k_L = 0.9$, $\rho_L = 1$, $k_g = 0.1$, $\rho_g = 1$, $s = 100$.



(a) Copper bunny, $k_L = 0$, $k_g = 1$, $\vec{F}_0 = \vec{\rho} = [0.95, 0.64, 0.54]$, $\alpha_b = 0.4$.



(b) Lambertian bunny, $k_L = 1$, $k_g = 0$, $\vec{\rho} = [0.95, 0.64, 0.54]$.



(c) Gold bunny, $k_L = 0.2$, $k_g = 0.8$, $\vec{F}_0 = \vec{\rho} = [1, 0.78, 0.34]$, $\alpha_b = 0.3$.



(d) Gold bunny with multicoloured lights.

Figure 7: Stanford bunnies with Cook–Torrance shading.

+1 point

2.4 Physically-based reflection models

While the phenomenological Phong model of Task 2.3 is flexible enough to allow for modelling several different materials, everything tends to look rather like plastic. It falls short on realistically rendering metallic and ceramic surfaces; and in particular, it is not physically correct. In this task you will also implement a physically-based reflection model.

Add a new function to your fragment shader that implements Cook–Torrance scattering, instead of Blinn–Phong. The result might look like Figure 7.

Refer to Equations (9.34), (9.35), (9.16) in the RTR book for this, as well as lecture notes from Lecture 4.

The full Cook–Torrance BRDF is, just as for Blinn–Phong, a weighted sum of a diffuse and a glossy component:

$$f_r = k_L(f_{diff}) + k_g(f_{spec}), \tag{3}$$

with $k_L + k_g \leq 1$. The diffuse term is the same Lambertian term that you have used before. The difference from the Blinn–Phong model is a more correct glossy component f_{spec} that is based on a microfacet model. The glossy component is on the form given by (9.34) in the book, and requires the following:

- The Fresnel factor F is given by (9.16).

  Note that $\vec{n}$ in (9.16) should be substituted by the halfway vector $\vec{h}$, when called from (9.34). F_0 in (9.16) is a colour with potentially different values for r/g/b, as specified in, e.g., Tables 9.1 and 9.2.

- The microfacet distribution D is given by (9.35).

  The notation in the book might require a bit of explanation. Note that the term $\chi^+(\vec{n} \cdot \vec{m})$ is simply 1 if the dot product is positive and 0 otherwise. This is the same test for negative dot products that you implemented for the Lambert-shaded cube. The microfacet normal $\vec{m}$ in (9.35) is the same as the half-way vector $\vec{h}$, since only microfacets with that normal contribute to the colour in this model. (Finally, you might find the exponent expressed with a tan function in some other sources — including the original paper by Cook and Torrance. However, the expression in the book is equivalent, from trigonometric identities.)

- A geometric attenuation function G. There is a few versions of G stated in the RTR book, but in Cook's original paper it is stated as

$$G = \min\Big(1, \frac{2(\vec{n} \cdot \vec{h})(\vec{n} \cdot \vec{\omega}_o)}{(\vec{\omega}_o \cdot \vec{h})}, \frac{2(\vec{n} \cdot \vec{h})(\vec{n} \cdot \vec{\omega}_i)}{(\vec{\omega}_o \cdot \vec{h})}\Big). \tag{4}$$

In the Cook–Torrance model, the specular colour is set using the Fresnel reflectance F_0, which is uncoloured with low values for dielectrics (e.g., plastic and wood) but coloured with higher values for metals. (See Tables 9.1 and 9.2.) For metals, it makes sense to use the same diffuse albedo $\vec{\rho}$ and specular tint $\vec{F}_0$. For dielectrics, they will be different.

Instead of a "magic number" s to control the glossiness, Cook–Torrance uses a roughness factor α_b that governs the distrubution of the microfacets, in the normal distribution function D. Experiment with different values of α_b. What happens if you set it close to 0, or close to 1, or above?

+1 point

2.5 Anisotropic BRDFs

In this task, you should create an *anisotropic* version of your Blinn–Phong or Cook–Torrance BRDF from the previous labs. That is, a BRDF that where the scattering is different along different directions. A typical example of when this happens in real life is a brushed-metal surface of pots and pans. See also Figure 8.

In order to do this, you need to specify a coordinate frame *for each point* that is tangential to the surface at the point. In Lab 3, you will see ways of

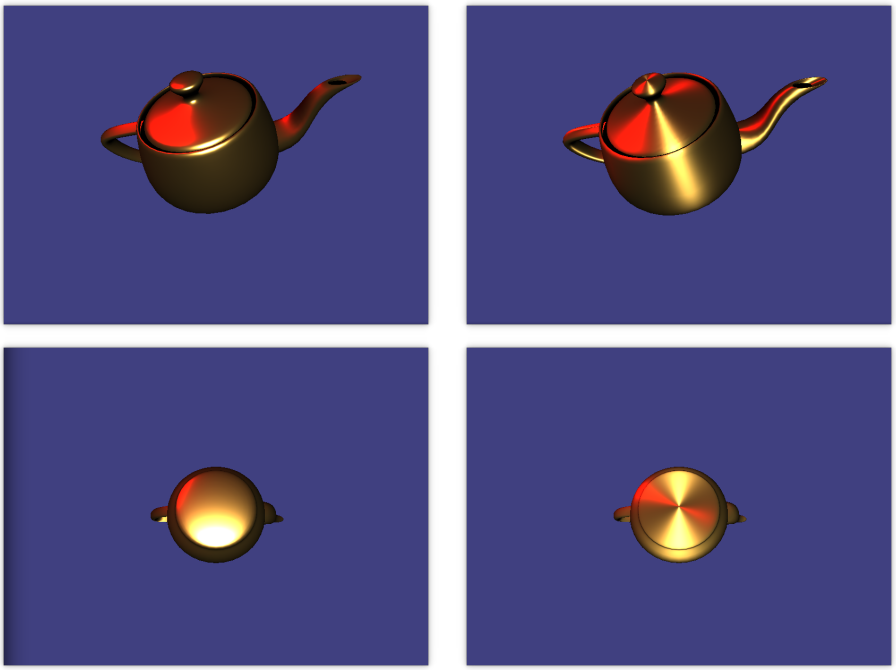


Figure 8: Cook–Torrance shaded teapot from Task 2.5. Left: *isotropic* shading (roughness 0.15). Right: *anisotropic* shading, with roughness 0.15 in the direction around the circumference of the teapot, and 0.7 in the top–bottom direction.

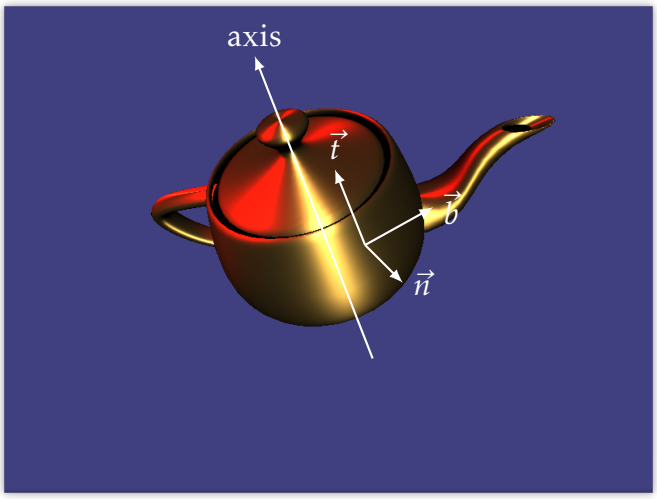


Figure 9: Computing a tangent space for anisotropic shading. The vectors $\vec{t}$ and $\vec{b}$ are both perpendicular to the normal vector (which points out from the surface).

"wrapping" a user-defined image around the object to define the local surface directions in a flexible way. For now, we'll do it in a simpler way, that works well for more-or-less cylindrical objects like the teapot in Figure 8. You can define a central axis, and define the tangent coordinate frame such that one direction goes *along* the axis (in the top–bottom direction) and the other direction goes *around* the axis (around the teapot). One way to do that is as follows:

1. In the vertex shader, define your axis. (For the teapot, the axis should go along the z axis.)
2. Transform the axis vector in the same way as you do with the position attributes (i e, with the model and view matrix).
3. Pass the vector as `out` to the fragment shader.
4. For each point (in the fragment shader), compute the tangent and bitangent vectors, which shall be tangent to the surface at the point, with the tangent oriented along your main axis, and the bitangent perpendicular to it. In other words, what you will do here is to create a coordinate frame with three perpendicular vectors, which defines the local surface direction, so that you can apply one roughness parameter along one direction and another roughness parameter around the other. The bitangent vector can be computed as the cross product of the normal vector and the axis vector. (Remember to only use normalised vectors in the cross product.) The tangent vector is the cross product of the bitangent and the normal.
5. With these vectors set up, you can now implement an anisotropic NDF, like (9.53) in the RTR4 book.
6. Update your BRDF to use different roughness values along he tangent vs bitangent directions.

Your result should look something like Figure 8. In particular, note the radial pattern when viewing the teapot from below.

For the additions in the vertex shader, you can simply modify your existing `lab2_vs.glsl` if you don't want to create a new file.

+2 points

2.6 Mimicking real materials

Provide a systematic *evaluation and comparison* of Blinn–Phong, Lambert, and Cook–Torrance shading. You can also evaluate and compare different variants of the normal distribution function from your Cook–Torrance above, and replace the Beckmann distribution from (9.35) with the GGX distribution. Possibly, try another microfacet model such as Oren–Nayar. How does it affect the appearance of your objects (and why)?

Cover the following points in your report.

- Systematically explore different parameters for the models and report on the results.
- Try to mimic the appearance of some real-world objects (see Figure 10 for inspiration.) What are good shader models for these materials? What are good parameters?
- Discuss the limits of the models, in terms of which materials they can mimic.
- What are sensible ranges for the parameters? (Sensible as in meaningful for representing a physical material.)



Figure 10: A few examples of materials with different scattering properties. Metallic and glossy dark plastic dice, a paper crane; a velvet dress and a velvet ball. How can these be modelled in a shader? (For the velvet objects, note that the highlights are at an oblique angle, rather than centred. This is maybe most visible if you compare the velvet ball to the more glossy Christmas balls.)